

Assignment 8: Time Series Analysis

Josh Salzberg

Fall 2025

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on generalized linear models.

Directions

1. Rename this file `<FirstLast>_A08_TimeSeries.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.

Set up

1. Set up your session:
 - Check your working directory
 - Load the tidyverse, lubridate, zoo, and trend packages
 - Set your ggplot theme

```
library(here)
```

```
## here() starts at /home/guest/project/EDE_Fall2025
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.1      v tibble    3.2.1
## v lubridate  1.9.3      v tidyr     1.3.1
## v purrr      1.0.2
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(lubridate)
library(agricolae)
library(zoo)
```

```
##
## Attaching package: 'zoo'
##
## The following objects are masked from 'package:base':
##
##   as.Date, as.Date.numeric
```

```
library(trend)

here()
```

```
## [1] "/home/guest/project/EDE_Fall2025"
```

```
my_theme <- theme_bw() +
  theme(
    plot.background = element_rect(fill = "azure"),
    panel.background = element_rect(fill = "linen"),
    legend.background = element_rect(fill = "antiquewhite"),

    # Modifying legend.position
    legend.position = 'top',

  )
theme_set(my_theme)
```

2. Import the ten datasets from the Ozone_TimeSeries folder in the Raw data folder. These contain ozone concentrations at Garinger High School in North Carolina from 2010-2019 (the EPA air database only allows downloads for one year at a time). Import these either individually or in bulk and then combine them into a single dataframe named **GaringerOzone** of 3589 observation and 20 variables.

```
#1
RawOzoneList <- c(here('Data','Raw','Ozone_TimeSeries','EPAair_03_GaringerNC2010_raw.csv'),
  here('Data','Raw','Ozone_TimeSeries','EPAair_03_GaringerNC2011_raw.csv'),
  here('Data','Raw','Ozone_TimeSeries','EPAair_03_GaringerNC2012_raw.csv'),
  here('Data','Raw','Ozone_TimeSeries','EPAair_03_GaringerNC2013_raw.csv'),
  here('Data','Raw','Ozone_TimeSeries','EPAair_03_GaringerNC2014_raw.csv'),
  here('Data','Raw','Ozone_TimeSeries','EPAair_03_GaringerNC2015_raw.csv'),
  here('Data','Raw','Ozone_TimeSeries','EPAair_03_GaringerNC2016_raw.csv'),
  here('Data','Raw','Ozone_TimeSeries','EPAair_03_GaringerNC2017_raw.csv'),
  here('Data','Raw','Ozone_TimeSeries','EPAair_03_GaringerNC2018_raw.csv'),
  here('Data','Raw','Ozone_TimeSeries','EPAair_03_GaringerNC2019_raw.csv'))

# Using For Loop Method from https://www.geeksforgeeks.org/r-language/how-to-read-many-files-in-r-with-

OzoneRaw <- list()

for (i in RawOzoneList) {
```

```

rawdata <- read.csv(i,stringsAsFactors = T)
OzoneRaw[[i]] <- rawdata
}

OzoneCompiled <- bind_rows(OzoneRaw)

```

Wrangle

3. Set your date column as a date class.
4. Wrangle your dataset so that it only contains the columns Date, Daily.Max.8.hour.Ozone.Concentration, and DAILY_AQI_VALUE.
5. Notice there are a few days in each year that are missing ozone concentrations. We want to generate a daily dataset, so we will need to fill in any missing days with NA. Create a new data frame that contains a sequence of dates from 2010-01-01 to 2019-12-31 (hint: `as.data.frame(seq())`). Call this new data frame Days. Rename the column name in Days to “Date”.
6. Use a `left_join` to combine the data frames. Specify the correct order of data frames within this function so that the final dimensions are 3652 rows and 3 columns. Call your combined data frame GaringerOzone.

```

# 3
class(OzoneCompiled$Date)

```

```
## [1] "factor"
```

```

OzoneCompiled$Date <- mdy(OzoneCompiled$Date)
class(OzoneCompiled$Date)

```

```
## [1] "Date"
```

```

# 4
OzoneWrangle <- OzoneCompiled %>%
  select(Date, Daily.Max.8.hour.Ozone.Concentration, DAILY_AQI_VALUE)

```

```

# 5
remove(DateFrame)

```

```
## Warning in remove(DateFrame): object 'DateFrame' not found
```

```

DateFrame <- as.data.frame(seq(as.Date('2010-01-01'), as.Date('2019-12-31'), by = 1), optional = T)
colnames(DateFrame)[1] <- "Date"
# 6
GaringerOzone <- left_join(y=OzoneWrangle, x=DateFrame)

```

```
## Joining with 'by = join_by(Date)'
```

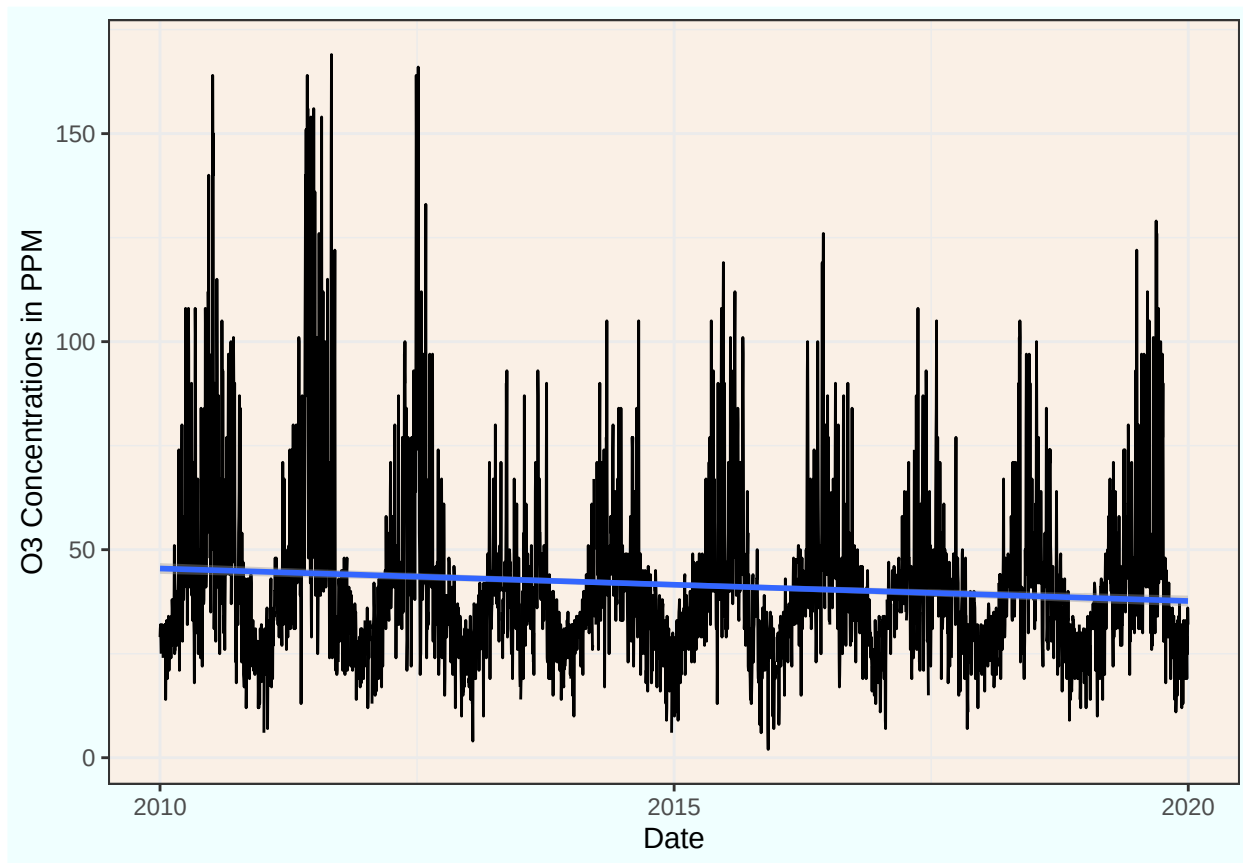
Visualize

7. Create a line plot depicting ozone concentrations over time. In this case, we will plot actual concentrations in ppm, not AQI values. Format your axes accordingly. Add a smoothed line showing any linear trend of your data. Does your plot suggest a trend in ozone concentration over time?

```
#7
ggplot(GaringerOzone, aes(x=Date, y=DAILY_AQI_VALUE))+
  geom_line()+
  geom_smooth(method="lm")+
  ylab("O3 Concentrations in PPM")+
  my_theme
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

```
## Warning: Removed 63 rows containing non-finite outside the scale range
## ('stat_smooth()').
```



Answer: the plot suggests that there's perhaps a slight decline to the data over time

Time Series Analysis

Study question: Have ozone concentrations changed over the 2010s at this station?

8. Use a linear interpolation to fill in missing daily data for ozone concentration. Why didn't we use a piecewise constant or spline interpolation?

```
#8
GaringerOzone_Interpolated <- GaringerOzone %>%
  mutate(DailyOzoneInterpo = na.approx(Daily.Max.8.hour.Ozone.Concentration))
```

Answer: Linear interpolation is convenient because it does not result in changing the overall mean of the data.

9. Create a new data frame called `GaringerOzone.monthly` that contains aggregated data: mean ozone concentrations for each month. In your pipe, you will need to first add columns for year and month to form the groupings. In a separate line of code, create a new Date column with each month-year combination being set as the first day of the month (this is for graphing purposes only)

```
#9
GaringerOzone.monthly <- GaringerOzone_Interpolated %>%
  mutate(Month = month(Date), Year = year(Date), DailyOzone = DailyOzoneInterpo) %>%
  group_by(Year, Month) %>%
  summarize(MeanOzone = mean(DailyOzone))
```

```
## 'summarise()' has grouped output by 'Year'. You can override using the
## '.groups' argument.
```

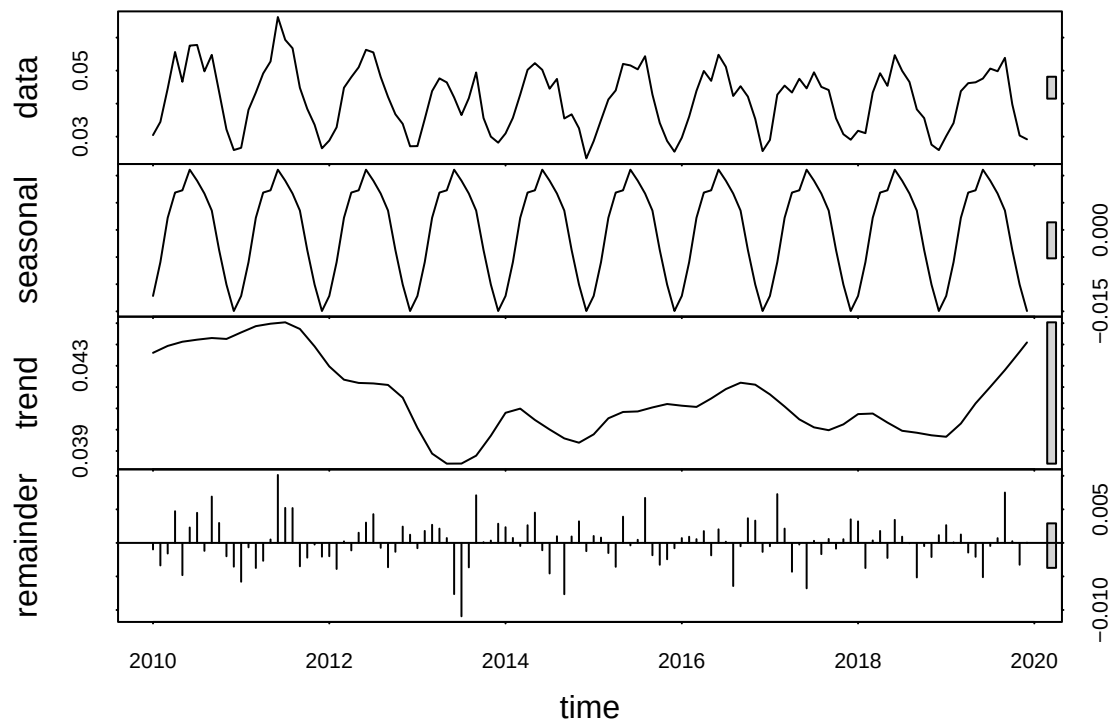
```
GaringerOzone.monthly <- GaringerOzone.monthly %>%
  mutate(NewDate = ymd(paste(Year, Month, "01")))
```

10. Generate two time series objects. Name the first `GaringerOzone.daily.ts` and base it on the dataframe of daily observations. Name the second `GaringerOzone.monthly.ts` and base it on the monthly average ozone values. Be sure that each specifies the correct start and end dates and the frequency of the time series.

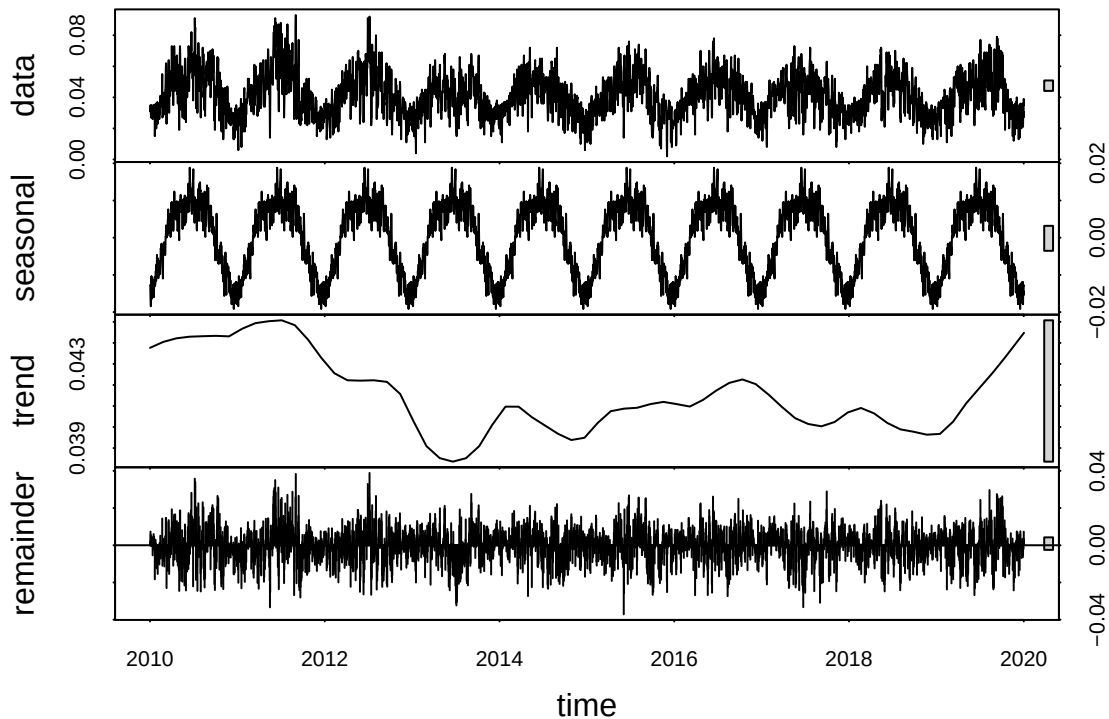
```
#10
f_month <- month(first(GaringerOzone.monthly$NewDate))
f_year <- year(first((GaringerOzone.monthly$NewDate)))
GaringerOzone.monthly.ts <- ts(GaringerOzone.monthly$MeanOzone,
  start=c(f_year,f_month),
  frequency=12)
f_month2 <- month(first(GaringerOzone_Interpolated$Date))
f_year2 <- year(first((GaringerOzone_Interpolated$Date)))
GaringerOzone.daily.ts <- ts(GaringerOzone_Interpolated$DailyOzoneInterpo,
  start=c(f_year2,f_month2),
  frequency=365)
```

11. Decompose the daily and the monthly time series objects and plot the components using the `plot()` function.

```
#11
DecompMonthly <- stl(GaringerOzone.monthly.ts, s.window = "periodic")
plot(DecompMonthly)
```



```
DecompDaily <- stl(GaringerOzone.daily.ts, s.window="periodic")
plot(DecompDaily)
```



12. Run a monotonic trend analysis for the monthly Ozone series. In this case the seasonal Mann-Kendall is most appropriate; why is this?

#12

```
GaringerDailyTrend <- Kendall::SeasonalMannKendall(GaringerOzone.daily.ts)
summary(GaringerDailyTrend)
```

```
## Score = -739 , Var(Score) = 45223.67
## denominator = 16213.86
## tau = -0.0456, 2-sided pvalue =0.00051075
```

```
GaringerMonthlyTrend <- Kendall::SeasonalMannKendall(GaringerOzone.monthly.ts)
summary(GaringerMonthlyTrend)
```

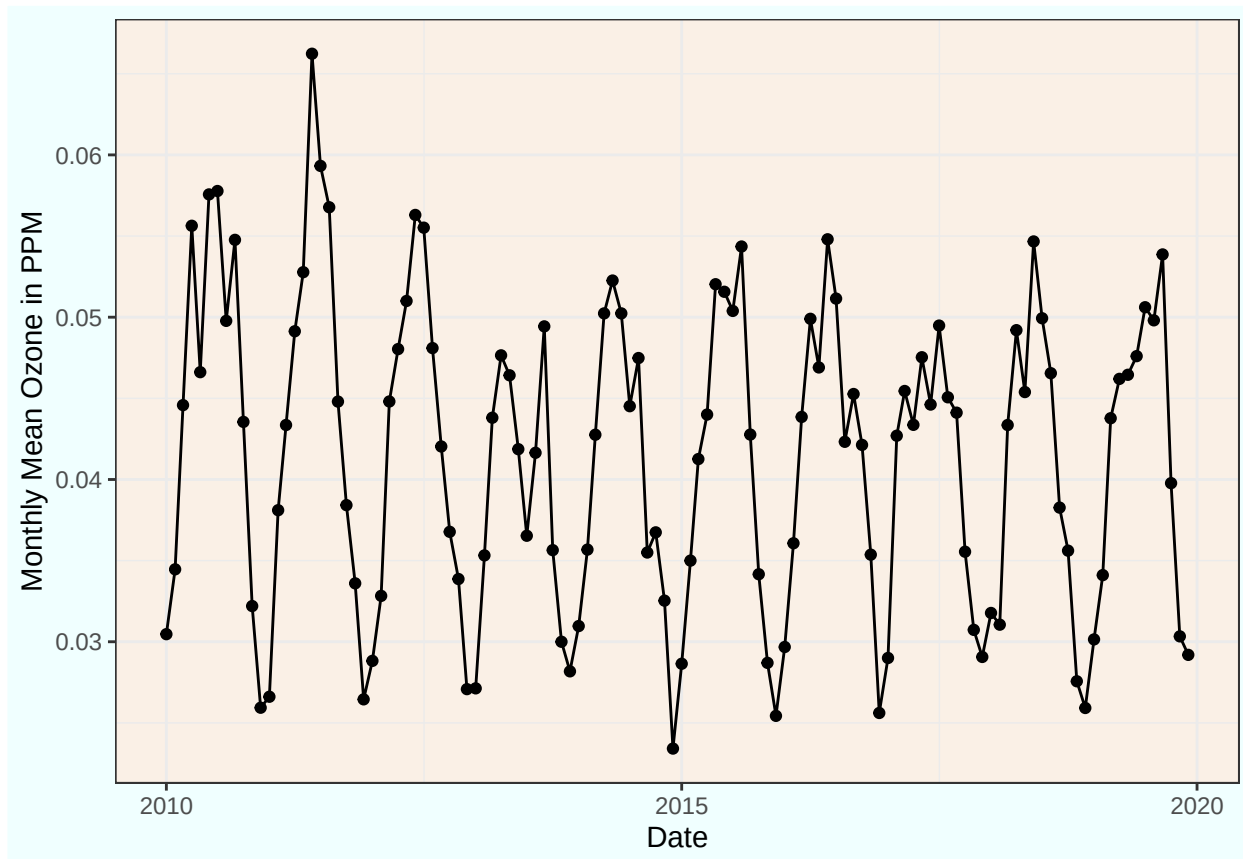
```
## Score = -77 , Var(Score) = 1499
## denominator = 539.4972
## tau = -0.143, 2-sided pvalue =0.046724
```

Answer: we can assume that seasons have predictable impacts on ozone levels.

13. Create a plot depicting mean monthly ozone concentrations over time, with both a `geom_point` and a `geom_line` layer. Edit your axis labels accordingly.

13

```
ggplot(GaringerOzone.monthly, aes(x=NewDate,y=MeanOzone))+  
  geom_point()+  
  geom_line()+  
  xlab("Date")+  
  ylab("Monthly Mean Ozone in PPM")+  
  my_theme
```



14. To accompany your graph, summarize your results in context of the research question. Include output from the statistical test in parentheses at the end of your sentence. Feel free to use multiple sentences in your interpretation.

Answer: The data indicates consistent and statistically significant seasonality, with peaks in the summer and nadirs in the winter (2 sided p value of 0.046)

15. Subtract the seasonal component from the `GaringerOzone.monthly.ts`. Hint: Look at how we extracted the series components for the `EnoDischarge` on the lesson Rmd file.
16. Run the Mann Kendall test on the non-seasonal Ozone monthly series. Compare the results with the ones obtained with the Seasonal Mann Kendall on the complete series.

#15

```
GaringerMonthly_Components <- as.data.frame(DecompMonthly$time.series[,1:3])
```



```

GaringerMonthly_Components <- mutate(GaringerMonthly_Components,
  Observed = GaringerOzone.monthly$MeanOzone,
  Date = GaringerOzone.monthly$NewDate)

#16
f_month3 <- month(first(GaringerMonthly_Components$Date))
f_year3 <- year(first((GaringerMonthly_Components$Date)))
GaringerMonthly.Components.ts <- ts(GaringerMonthly_Components$trend,
  start=c(f_year3,f_month3),
  frequency=365)
GaringerMonthlyTrend_Components <- Kendall::MannKendall(GaringerMonthly.Components.ts)
summary(GaringerMonthlyTrend_Components)

## Score = -1922 , Var(Score) = 194366.7
## denominator = 7140
## tau = -0.269, 2-sided pvalue =1.3168e-05

```

Answer: The results indicate that the trend, isolated from the seasonal components, is much more statistically significant, resulting in a 2-sided p value of 0.0000013168